



Universidad Argentina de la Empresa

FUNDAMENTOS DE PROGRAMACION - 563075

Introduccion a la Algoritmia

TRABAJO PRACTICO GRUPAL Y OBLIGATORIO

PARTE 2 - LOLE KARTING

Docente: Lic. Christian G. Schendel

Numero de Grupo: 6

Integrantes

Leguizamon Jose

Volpe Santiago

Martino Tiago

Martinez Victor

Molinari Matias

1. Introduccion

Este trabajo es la Parte 2 del proyecto de Lole Karting, un sistema de ventas para un local de karting. El programa toma los datos del cliente, le muestra los servicios disponibles, calcula el total con sus descuentos y recargos, emite una factura por cada venta y al final muestra una matriz con todas las ventas, sus estadísticas y un buscador.

Para esta segunda parte partimos del código de la Parte 1 y aplicamos todas las correcciones que indicó el docente en su devolución, sumando los nuevos requisitos pedidos. Además reorganizamos todo el programa en funciones para que sea más ordenado y fácil de mantener. Solo se utilizaron temas vistos en la cursada (listas, listas paralelas, matrices, funciones, módulo random, ordenamiento burbuja, condicionales y bucles).

2. Correcciones de la Parte 1

La siguiente tabla resume cada observación del docente y cómo fue resuelta en esta entrega:

Observacion del docente	Como lo resolvimos
Linea 73: si el menor no llega a la edad minima no se debe volver a consultar; mostrar un mensaje e interrumpir la facturacion.	Si la edad es menor a 10, la funcion <code>pedir_datos_cliente()</code> muestra el mensaje y devuelve <code>None</code> . Esa venta se cancela y se pasa a la siguiente.
Linea 164: <code>while</code> no lleva <code>else</code> .	Se revisaron todos los <code>while</code> del programa. Ninguno usa <code>else</code> .
Linea 277: las estadísticas estaban pensadas solo para 3 ventas. "Y si les pido 100?"	Se eliminaron las variables fijas (<code>total_venta1</code> , 2, 3). Ahora todo se guarda en una matriz y se calcula con bucles. Cambiando <code>CANTIDAD_VENTAS</code> funciona con cualquier cantidad.
Linea 100 en adelante: mostrar los precios desde las variables creadas al inicio.	El catálogo (<code>mostrar_catálogo</code>) imprime los precios desde la lista <code>precios_servicios</code> , no como texto fijo.
Consultar al usuario si sus datos personales son correctos para que los confirme.	Al final de la carga, <code>pedir_datos_cliente()</code> muestra los datos y pide confirmación (s/n). Si responde 'n', se vuelven a cargar.

3. Requisitos nuevos de la Parte 2

Estos son los puntos nuevos que se incorporaron, segun la consigna:

- Comentar el numero de grupo y los integrantes al inicio del programa: hecho en el encabezado del archivo .py.
- Declarar listas por cada grupo de valores: datos_negocio, nombres_servicios, precios_servicios, detalle_servicios, cantidades, subtotales, totales, facturas_usadas y la matriz_ventas.
- Definir funciones para los distintos procesos y reutilizarlas: 27 funciones en total. Por ejemplo sumar_lista() se reutiliza en tres lugares y calcular_subtotal() se reutiliza dentro de calcular_subtotales().
- Numero de factura aleatorio y sin repetir: la funcion generar_factura() usa random.randint(1000, 9999) y valida con un bucle que el numero no este ya usado.
- Matriz 3x4 con factura, dos datos del cliente (nombre y apellido) y el monto, ordenada por factura de menor a mayor: se arma matriz_ventas con [factura, nombre, apellido, total] (3 ventas x 4 columnas) y se ordena con ordenar_matriz() (metodo burbuja). El DNI se guarda en una lista paralela a la matriz, asi se puede buscar por DNI sin alterar el formato 3x4.
- Buscador final con busqueda lineal (Clase 13): busqueda_lineal() recorre una lista posicion por posicion y devuelve la posicion encontrada o -1. buscar_venta() la aplica sobre tres listas (nombres, apellidos y dnis) y repite hasta encontrar coincidencia.

4. Explicacion de las funciones

El programa esta organizado en 27 funciones agrupadas por su rol. El programa principal quedo reducido a solo dos llamadas: una que procesa todas las ventas y otra que muestra el cierre.

4.1 Funciones de validacion

Piden un dato por teclado y lo validan con un bucle while, repitiendo hasta que sea correcto.

Funcion	Que hace
pedir_texto(mensaje)	Pide un texto y valida que tenga al menos 2 caracteres. Se usa para nombres y apellidos.
pedir_dni(mensaje)	Pide un DNI y valida que tenga 7 u 8 digitos.
pedir_si_no(mensaje)	Pide una respuesta y valida que sea 's' o 'n'.
pedir_numero(mensaje, minimo, maximo)	Pide un numero entero y valida que este dentro del rango indicado.

4.2 Funciones de calculo

Hacen las operaciones aritmeticas del programa. Son funciones cortas y reutilizables.

Funcion	Que hace
calcular_subtotal(cantidad, precio)	Multiplca cantidad por precio y devuelve el subtotal.

Funcion	Que hace
<code>calcular_total(subtotal, descuento, recargo)</code>	Aplica descuento y recargo sobre el subtotal y devuelve el total final.
<code>calcular_porcentaje(parte, total)</code>	Calcula el porcentaje con la formula $(parte * 100) / total$.
<code>sumar_lista(lista)</code>	Recorre una lista y devuelve la suma de todos sus elementos. Reutilizada en varios lugares.
<code>maximo_lista(lista)</code>	Recorre una lista y devuelve el valor mas alto (inicializando en -9999).
<code>calcular_subtotales(cantidades)</code>	Arma la lista de subtotales reutilizando <code>calcular_subtotal()</code> .

4.3 Funciones de proceso de una venta

Se encargan de cada parte del proceso de venta. `procesar_venta()` las coordina a todas.

Funcion	Que hace
<code>generar_factura(facturas_usadas)</code>	Genera un numero de factura aleatorio y valida que no se repita.
<code>ordenar_matriz(matriz, dnis)</code>	Ordena la matriz por numero de factura de menor a mayor (metodo burbuja), moviendo en paralelo la lista de DNIs.
<code>mostrar_catalogo()</code>	Muestra el menu de servicios tomando los precios desde las listas.
<code>pedir_datos_cliente()</code>	Pide y valida los datos del cliente (y del adulto si es menor). Pide confirmacion. Devuelve None si el menor no llega a la edad minima.
<code>pedir_servicios()</code>	Pide los servicios elegidos y devuelve la lista de cantidades.
<code>procesar_descuento(cantidades, subtotales)</code>	Calcula el descuento del 20% si hay paquete de cumpleaños anticipado.
<code>procesar_recargo(subtotal)</code>	Calcula el recargo del 15% si hubo daños.
<code>elegir_medio_pago()</code>	Muestra los medios de pago y devuelve el elegido.
<code>mostrar_factura(...)</code>	Muestra la factura completa de una venta.
<code>mostrar_titulo_venta(numero, total_ventas)</code>	Muestra el encabezado de cada proceso de venta.
<code>procesar_venta(numero, total_ventas, facturas_usadas)</code>	Orquesta una venta completa. Devuelve la fila 3x4 [factura, nombre, apellido, total] y el DNI por separado (para la lista paralela).

4.4 Funciones de cierre

Se ejecutan despues de procesar todas las ventas: ordenan y muestran la matriz, las estadísticas y el buscador.

Funcion	Que hace
<code>mostrar_matriz(matriz)</code>	Muestra la matriz 3x4 ya ordenada (factura, nombre, apellido, total).
<code>mostrar_estadisticas(matriz)</code>	Calcula y muestra total, promedio, maximo y porcentajes.
<code>busqueda_lineal(lista, dato)</code>	Recorre una lista posicion por posicion. Devuelve la posicion si encuentra el dato, o -1 si no lo encuentra. Implementacion de Clase 13.
<code>buscar_venta(matriz, dnis)</code>	Arma listas de nombres, apellidos y dnis, aplica <code>busqueda_lineal</code> sobre cada una y muestra el resultado. Repite hasta encontrar.
<code>mostrar_despedida()</code>	Muestra el mensaje final del negocio.
<code>mostrar_cierre(matriz, dnis)</code>	Ordena y muestra la matriz, estadísticas, buscador y despedida.
<code>procesar_todas_las_ventas(cantidad_ventas)</code>	Procesa todas las ventas y devuelve la matriz 3x4 y la lista paralela de DNIs.

5. Guia para la defensa

Estos son los temas vistos en clase que aparecen en el programa, por si el docente pregunta donde se usa cada uno:

- Listas: se usan para los datos del negocio, los servicios (listas paralelas) y los acumuladores. Visto en Clase 9 y 10.
- Listas paralelas: nombres_servicios, precios_servicios y detalle_servicios comparten la misma posicion para cada servicio. Tambien la lista de DNIs es paralela a la matriz: la venta de la fila i tiene su DNI en dnis[i]. Visto en Clase 10.
- Matriz 3x4 (lista de listas): matriz_ventas, donde cada fila es una venta con [factura, nombre, apellido, total]. Recorrida con matriz[i][columna]. Visto en Clase 12.
- Funciones con return: todas las funciones de calculo devuelven un valor; las de mostrar son procedimientos (no retornan). Visto en Clases 9 a 12.
- Modulo random: random.randint() para el numero de factura. Visto en Clase 11 y 12.
- Ordenamiento burbuja: en ordenar_matriz(), comparando pares y usando una variable auxiliar (aux). Visto en Clase 12.
- Maximo: maximo_lista() inicializa en -9999 y compara. Visto en Clase 8 y 11.
- Promedio y porcentaje: con las formulas suma/cantidad y (parte*100)/total. Visto en Clase 6.
- While de validacion: todas las funciones pedir_* repiten hasta que el dato es valido. Visto en Clase 7.
- Busqueda lineal (Clase 13): busqueda_lineal() recorre con for i in range(len(lista)) y devuelve la posicion o -1. Se llama tres veces en buscar_venta(): sobre nombres, apellidos y DNIs.

6. Codigo fuente completo

A continuacion se incluye el codigo completo del programa (archivo lole_karting_p2.py):

```
# =====
# FUNDAMENTOS DE PROGRAMACION - 563075
# TRABAJO PRACTICO GRUPAL Y OBLIGATORIO - PARTE 2
# Docente: Lic. Christian G. SCHENDEL
#
# Nro. de Grupo: 6
# Integrantes:
#   - Leguizamon Jose
#   - Volpe Santiago
#   - Martino Tiago
#   - Martinez Victor
#   - Molinari Matias
#
# LOLE KARTING - Sistema de Ventas
# =====

import random # Modulo random (visto en clase) para el nro de factura

# =====
# DATOS DEL NEGOCIO (guardados en una lista)
# =====
# Posiciones: 0 nombre, 1 cuit, 2 direccion, 3 telefono, 4 instagram, 5 horarios
datos_negocio = [
    "LOLE KARTING",
    "30-71234567-9",
    "Av. Cnel. Roca 3850, Lugano, CABA",
    "+54 11 4605-7890",
    "@lolekarting",
    "Lun-Vie 14:00-22:00 / Sab-Dom 10:00-22:00"
]

# =====
# DATOS DE LOS SERVICIOS (listas paralelas)
# =====
# Cada posicion de las tres listas representa un mismo servicio
nombres_servicios = ["Simulador de manejo", "Pista de karting", "Curso de manejo", "Paquete de cumpleaños"]
precios_servicios = [3500, 6000, 15000, 80000]
detalle_servicios = ["15 min", "10 min / 10 vueltas", "1 hora", "8 hs (pista + simuladores)"]

# =====
# SEPARADORES
# =====
separador = "===== "
linea = "-----"

# =====
# FUNCIONES DE VALIDACION
# =====

# Valida que un texto tenga al menos 2 caracteres. Devuelve el texto validado.
def pedir_texto(mensaje):
    texto = input(mensaje).title()
    while len(texto) < 2:
        print(" ERROR: Debe tener al menos 2 caracteres.")
        texto = input(mensaje).title()
    return texto

# Valida que un DNI tenga 7 u 8 digitos. Devuelve el DNI validado.
def pedir_dni(mensaje):
    dni = input(mensaje)
    while len(dni) < 7 or len(dni) > 8:
        print(" ERROR: El DNI debe tener 7 u 8 digitos.")
        dni = input(mensaje)
```

```

    return dni

# Valida una respuesta de tipo s/n. Devuelve "s" o "n".
def pedir_si_no(mensaje):
    resp = input(mensaje).lower()
    while resp != "s" and resp != "n":
        print(" ERROR: Ingrese 's' para si o 'n' para no.")
        resp = input(mensaje).lower()
    return resp

# Valida un numero entero dentro de un rango (incluye los limites).
def pedir_numero(mensaje, minimo, maximo):
    numero = int(input(mensaje))
    while numero < minimo or numero > maximo:
        print(" ERROR: Ingrese un numero entre", minimo, "y", maximo, ".")
        numero = int(input(mensaje))
    return numero

# =====
# FUNCIONES DE OPERACIONES ARITMETICAS
# =====

# Calcula el subtotal multiplicando cantidad por precio.
def calcular_subtotal(cantidad, precio):
    return cantidad * precio

# Calcula el total final aplicando descuento y recargo sobre el subtotal.
def calcular_total(subtotal, descuento, recargo):
    return round(subtotal - descuento + recargo, 2)

# Calcula un porcentaje: (parte * 100) / total.
def calcular_porcentaje(parte, total):
    return round((parte * 100) / total, 2)

# Recorre una lista de montos y devuelve la suma de todos.
def sumar_lista(lista):
    suma = 0
    for i in range(len(lista)):
        suma = suma + lista[i]
    return round(suma, 2)

# Recorre una lista de montos y devuelve el valor maximo.
def maximo_lista(lista):
    maximo = -9999
    for i in range(len(lista)):
        if lista[i] > maximo:
            maximo = lista[i]
    return maximo

# Recibe la lista de cantidades y devuelve la lista de subtotales (lista paralela).
def calcular_subtotales(cantidades):
    subtotales = [0, 0, 0, 0]
    for i in range(len(cantidades)):
        subtotales[i] = calcular_subtotal(cantidades[i], precios_servicios[i])
    return subtotales

# =====
# FUNCION DE NUMERO DE FACTURA ALEATORIO Y SIN REPETIR
# =====

# Genera un nro de factura aleatorio que no este ya en la lista de usados.
def generar_factura(facturas_usadas):
    nro = random.randint(1000, 9999)
    repetido = True
    while repetido == True:
        repetido = False
        for i in range(len(facturas_usadas)):
            if facturas_usadas[i] == nro:
                repetido = True

```

```

        if repetido == True:
            nro = random.randint(1000, 9999)
        return nro

# =====
#   FUNCION DE ORDENAMIENTO (METODO BURBUJA)
# =====

# Ordena la matriz por nro de factura (columna 0) de menor a mayor.
# Ordena en paralelo la lista de DNIs para que cada DNI siga junto a su venta.
def ordenar_matriz(matriz, dnis):
    for i in range(len(matriz)):
        for j in range(len(matriz) - i - 1):
            if matriz[j][0] > matriz[j + 1][0]:
                # Intercambiamos la fila de la matriz
                aux = matriz[j]
                matriz[j] = matriz[j + 1]
                matriz[j + 1] = aux
                # Intercambiamos el DNI en la misma posicion (lista paralela)
                aux_dni = dnis[j]
                dnis[j] = dnis[j + 1]
                dnis[j + 1] = aux_dni

# =====
#   FUNCION QUE MUESTRA EL CATALOGO DE SERVICIOS
# =====

# Muestra el menu de servicios tomando los precios desde las listas.
def mostrar_catalogo():
    print(separador)
    print("          SERVICIOS DISPONIBLES - LOLE KARTING")
    print(separador)
    for i in range(len(nombres_servicios)):
        print(" ", i + 1, ".", nombres_servicios[i], " $", precios_servicios[i], " |",
detalle_servicios[i])
    print(" DESCUENTOS:")
    print("    20% en paquete de cumpleaños abonando con mas de 7 dias de anticipacion")
    print(" RECARGOS:")
    print("    15% por daños al karting o al equipamiento")
    print(" MEDIOS DE PAGO:")
    print("    Efectivo | Tarjeta credito/debito | Billetera virtual (MP/Uala)")
    print(separador)
    print(" Ingrese 0 para finalizar la seleccion.")
    print(separador)

# =====
#   FUNCION QUE PIDE Y VALIDA LOS DATOS DEL CLIENTE
# =====

# Pide los datos del cliente y los devuelve dentro de una lista.
# La lista tiene: [nombre, apellido, dni, nombre_adulto, apellido_adulto, dni_adulto]
# Devuelve None si el menor no cumple la edad minima (se interrumpe la venta).
def pedir_datos_cliente():
    datos_ok = "n"
    while datos_ok == "n":
        print("--- DATOS DEL CLIENTE ---")
        nombre = pedir_texto("Nombre del cliente: ")
        apellido = pedir_texto("Apellido del cliente: ")
        dni = pedir_dni("DNI del cliente (7 u 8 digitos): ")
        es_mayor = pedir_si_no("El cliente es mayor de 18 años? (s/n): ")

        nombre_adulto = "-"
        apellido_adulto = "-"
        dni_adulto = "-"

        if es_mayor == "n":
            print("--- Verificacion de edad del menor ---")
            edad_menor = int(input("Edad del menor: "))
            # CORRECCION: si no llega a la edad minima NO se vuelve a preguntar,

```

```

        # se avisa y se interrumpe la venta devolviendo None.
        if edad_menor < 10:
            print(" El menor debe tener al menos 10 años para participar.")
            print(" Se cancela este proceso de venta.")
            return None
        print("-- Datos del adulto acompañante --")
        nombre_adulto = pedir_texto("Nombre del adulto: ")
        apellido_adulto = pedir_texto("Apellido del adulto: ")
        dni_adulto = pedir_dni("DNI del adulto (7 u 8 dígitos): ")

    # Confirmación de datos personales
    print("-- Confirme los datos ingresados --")
    print(" Cliente :", nombre, apellido)
    print(" DNI      :", dni)
    if es_mayor == "n":
        print(" Adulto  :", nombre_adulto, apellido_adulto)
        print(" DNI ad. :", dni_adulto)
    datos_ok = pedir_si_no("Son correctos los datos? (s/n): ")
    if datos_ok == "n":
        print(" Volvamos a cargar los datos...")

    return [nombre, apellido, dni, nombre_adulto, apellido_adulto, dni_adulto]

# =====
# FUNCION QUE PIDE LA SELECCION DE SERVICIOS
# =====

# Pide los servicios y devuelve una lista con las cantidades de cada uno.
# La posición coincide con las listas de servicios (0=simulador, 1=karting, etc).
def pedir_servicios():
    seleccion_valida = False
    cantidades = [0, 0, 0, 0]

    while seleccion_valida == False:
        cantidades = [0, 0, 0, 0]
        opcion_serv = -1
        while opcion_serv != 0:
            opcion_serv = pedir_numero(" Seleccione servicio (1-4) o 0 para terminar: ", 0, 4)
            if opcion_serv != 0:
                cantidad = pedir_numero(" Cuantas sesiones/unidades? ", 1, 999)
                # opcion 1 -> posición 0, opcion 2 -> posición 1, etc.
                cantidades[opcion_serv - 1] = cantidades[opcion_serv - 1] + cantidad
                print(" OK - Total", nombres_servicios[opcion_serv - 1], ":", cantidades[opcion_serv -
1])

        # Validar que se haya elegido al menos un servicio
        total_elegido = sumar_lista(cantidades)
        if total_elegido == 0:
            print(" ERROR: Debe seleccionar al menos un servicio.")
        else:
            seleccion_valida = True

    return cantidades

# =====
# FUNCIONES DE DESCUENTO, RECARGO Y MEDIO DE PAGO
# =====

# Calcula el descuento del 20% si hay paquete de cumpleaños y se abona anticipado.
# Recibe las cantidades y los subtotales. Devuelve el monto del descuento.
def procesar_descuento(cantidades, subtotales):
    descuento = 0
    # El paquete de cumpleaños está en la posición 3
    if cantidades[3] > 0:
        resp_desc = pedir_si_no("El paquete de cumpleaños se abona con mas de 7 días de anticipacion?
(s/n): ")
        if resp_desc == "s":
            descuento = round(subtotales[3] * 0.20, 2)
            print(" Descuento del 20% aplicado sobre el paquete de cumpleaños.")

```

```

        return descuento

# Calcula el recargo del 15% si hubo daños. Recibe el subtotal. Devuelve el recargo.
def procesar_recargo(subtotal):
    recargo = 0
    resp_daño = pedir_si_no("Hubo daños al karting o al equipamiento? (s/n): ")
    if resp_daño == "s":
        recargo = round(subtotal * 0.15, 2)
        print(" Recargo del 15% por daños aplicado.")
    return recargo

# Muestra los medios de pago, pide la opcion y devuelve el texto del medio elegido.
def elegir_medio_pago():
    print("MEDIOS DE PAGO:")
    print(" 1. Efectivo")
    print(" 2. Tarjeta de credito / debito")
    print(" 3. Billetera virtual (Mercado Pago / Uala)")
    opcion_pago = pedir_numero("Seleccione medio de pago (1-3): ", 1, 3)
    if opcion_pago == 1:
        medio_pago = "Efectivo"
    elif opcion_pago == 2:
        medio_pago = "Tarjeta de credito / debito"
    else:
        medio_pago = "Billetera virtual (Mercado Pago / Uala)"
    return medio_pago

# =====
# FUNCION QUE MUESTRA LA FACTURA
# =====

# Muestra la factura completa de una venta.
def mostrar_factura(nro_factura, cliente, cantidades, subtotales, subtotal, descuento, recargo, total,
medio_pago):
    print(separador)
    print("          F A C T U R A")
    print(separador)
    print(" Nro. de Factura : FC-", nro_factura)
    print(" *** DATOS DEL NEGOCIO ***")
    print(" Nombre      :", datos_negocio[0])
    print(" CUIT        :", datos_negocio[1])
    print(" Direccion   :", datos_negocio[2])
    print(" Telefono    :", datos_negocio[3])
    print(" Instagram   :", datos_negocio[4])
    print(" Horarios    :", datos_negocio[5])
    print(" *** DATOS DEL CLIENTE ***")
    print(" Nombre      :", cliente[0], cliente[1])
    print(" DNI         :", cliente[2])
    if cliente[3] != "-":
        print(" Adulto      :", cliente[3], cliente[4])
        print(" DNI adult.  :", cliente[5])
    print(" *** DETALLE DE SERVICIOS ***")
    for i in range(len(cantidades)):
        if cantidades[i] > 0:
            print(" ", nombres_servicios[i], ":", cantidades[i], "x $", precios_servicios[i], "= $",
subtotales[i])
    print(linea)
    print(" Subtotal          : $", subtotal)
    if descuento > 0:
        print(" Descuento 20% cumple : -$", descuento)
    if recargo > 0:
        print(" Recargo 15% daños    : +$", recargo)
    print(linea)
    print(" TOTAL A PAGAR      : $", total)
    print(" Medio de pago      :", medio_pago)
    print(separador)

# =====
# FUNCION QUE PROCESA UNA VENTA COMPLETA
# =====

```

```

# Muestra el titulo/encabezado de un proceso de venta.
def mostrar_titulo_venta(numero, total_ventas):
    print(separador)
    print("      LOLE KARTING  -  PROCESO DE VENTA", numero, "DE", total_ventas)
    print(separador)

# Procesa una venta de principio a fin.
# Recibe el numero de venta, el total de ventas y la lista de facturas ya usadas.
# Devuelve una lista con dos cosas:
# - la fila para la matriz 3x4: [nro_factura, nombre, apellido, total]
# - el dni del cliente (para guardarlo en la lista paralela de DNIs)
# Devuelve None si la venta se cancela (menor sin la edad minima).
def procesar_venta(numero, total_ventas, facturas_usadas):
    mostrar_titulo_venta(numero, total_ventas)

    # PASO 1: Datos del cliente
    cliente = pedir_datos_cliente()
    if cliente == None:
        return None

    # PASO 2: Catalogo y seleccion de servicios
    mostrar_catalogo()
    cantidades = pedir_servicios()

    # PASO 3: Calculos (subtotales, descuento, recargo y total)
    subtotales = calcular_subtotales(cantidades)
    subtotal = sumar_lista(subtotales)
    descuento = procesar_descuento(cantidades, subtotales)
    recargo = procesar_recargo(subtotal)
    medio_pago = elegir_medio_pago()
    total = calcular_total(subtotal, descuento, recargo)

    # PASO 4: Numero de factura aleatorio y sin repetir
    nro_factura = generar_factura(facturas_usadas)
    facturas_usadas.append(nro_factura)

    # PASO 5: Mostrar factura
    mostrar_factura(nro_factura, cliente, cantidades, subtotales, subtotal, descuento, recargo, total,
    medio_pago)

    # La fila de la matriz tiene 4 columnas (matriz 3x4 con 3 ventas):
    # 0 factura, 1 nombre, 2 apellido, 3 total
    fila = [nro_factura, cliente[0], cliente[1], total]
    # El DNI se guarda aparte, en una lista paralela a la matriz.
    dni = cliente[2]
    return [fila, dni]

# =====
#  FUNCION QUE MUESTRA LA MATRIZ DE VENTAS
#  =====

# Muestra la matriz 3x4 ya ordenada (factura, nombre, apellido, total).
def mostrar_matriz(matriz):
    print(separador)
    print("      MATRIZ DE VENTAS 3x4 (ordenada por factura)")
    print(separador)
    print(" FACTURA | NOMBRE      | APELLIDO    | TOTAL")
    print(linea)
    for i in range(len(matriz)):
        print(" FC-", matriz[i][0], " | ", matriz[i][1], " | ", matriz[i][2], " | $", matriz[i][3])
    print(separador)

# =====
#  FUNCION QUE MUESTRA LAS ESTADISTICAS
#  =====

# Calcula y muestra el resumen de las ventas (total, promedio, maximo y porcentajes).
def mostrar_estadisticas(matriz):

```

```

# Armamos una lista solo con los totales (columna 3) para las estadísticas
totales = []
for i in range(len(matriz)):
    totales.append(matriz[i][3])

total_recaudado = sumar_lista(totales)
promedio_ventas = round(total_recaudado / len(totales), 2)
maximo_venta = maximo_lista(totales)

print(separador)
print("      RESUMEN Y ESTADÍSTICAS DE LAS VENTAS")
print("      LOLE KARTING")
print(separador)
for i in range(len(matriz)):
    porc = calcular_porcentaje(matriz[i][3], total_recaudado)
    print("  Venta", i + 1, " (FC-", matriz[i][0], ") : $", matriz[i][3], "->", porc, "% del total")
print(linea)
print("  Total recaudado      : $", total_recaudado)
print("  Promedio por venta    : $", promedio_ventas)
print("  Maximo recaudado      : $", maximo_venta)
print(separador)

# =====
#  BUSQUEDA LINEAL (Clase 13)
# =====

# Recorre una lista posicion por posicion buscando el dato.
# Si algun elemento coincide, devuelve su posicion.
# Si no lo encuentra, devuelve -1.
def busqueda_lineal(lista, dato):
    posicion = -1
    for i in range(len(lista)):
        if lista[i].lower() == dato:
            posicion = i
    return posicion

# =====
#  FUNCION BUSCADOR DE CLIENTES
# =====

# Pide un dato por teclado y busca una venta por nombre, apellido o DNI.
# Arma listas separadas y aplica busqueda_lineal sobre cada una.
# Repite hasta que el dato coincida con alguna venta.
def buscar_venta(matriz, dnis):
    print("  BUSCADOR DE VENTAS")
    print("  Ingrese un nombre, apellido o DNI para buscar una venta.")

    # Armamos listas separadas de nombres y apellidos para aplicar busqueda lineal
    nombres = []
    apellidos = []
    for i in range(len(matriz)):
        nombres.append(matriz[i][1])
        apellidos.append(matriz[i][2])

    encontrado = False
    while encontrado == False:
        dato = input("  Dato a buscar: ").lower()

        # Aplicamos busqueda lineal sobre cada lista por separado
        pos = busqueda_lineal(nombres, dato)
        if pos == -1:
            pos = busqueda_lineal(apellidos, dato)
        if pos == -1:
            pos = busqueda_lineal(dnis, dato)

        # Si busqueda_lineal devolvio -1 en las tres, no se encontro
        if pos == -1:
            print("  No encontrado. Intente de nuevo.")
        else:

```

```

        print(linea)
        print(" VENTA ENCONTRADA")
        print(" Factura :", "FC-", matriz[pos][0])
        print(" Cliente :", matriz[pos][1], matriz[pos][2])
        print(" DNI      :", dnis[pos])
        print(" Total   : $", matriz[pos][3])
        print(linea)
        encontrado = True

# =====
# FUNCION DE DESPEDIDA
# =====

# Muestra el mensaje final de despedida del negocio.
def mostrar_despedida():
    print(separador)
    print(" Gracias por elegirnos! Seguin en", datos_negocio[4])
    print(" ", datos_negocio[0], "-", datos_negocio[2])
    print(separador)

# =====
# FUNCION DE CIERRE (matriz + estadisticas + buscador)
# =====

# Cierra el programa: si hubo ventas las ordena, muestra la matriz,
# las estadisticas, el buscador y la despedida.
# Recibe la matriz 3x4 y la lista paralela de DNIs.
def mostrar_cierre(matriz, dnis):
    if len(matriz) == 0:
        print(separador)
        print(" No se concreto ninguna venta.")
        print(separador)
    else:
        ordenar_matriz(matriz, dnis) # burbuja por nro de factura (matriz + dnis en paralelo)
        mostrar_matriz(matriz)
        mostrar_estadisticas(matriz)
        buscar_venta(matriz, dnis)
        mostrar_despedida()

# =====
# FUNCION QUE PROCESA TODAS LAS VENTAS
# =====

# Procesa todas las ventas. Devuelve una lista con dos cosas:
# - la matriz 3x4 con las ventas concretadas
# - la lista paralela de DNIs
def procesar_todas_las_ventas(cantidad_ventas):
    matriz_ventas = [] # cada fila: [factura, nombre, apellido, total]
    dnis_ventas = [] # lista paralela: el dni de cada venta
    facturas_usadas = [] # para controlar que no se repitan los nros

    nro_venta = 0
    while nro_venta < cantidad_ventas:
        resultado = procesar_venta(nro_venta + 1, cantidad_ventas, facturas_usadas)

        # Si la venta no se cancelo, guardamos la fila y el dni en paralelo
        if resultado != None:
            matriz_ventas.append(resultado[0]) # la fila 3x4
            dnis_ventas.append(resultado[1]) # el dni
        else:
            print(" Pasamos a la siguiente venta.")

        nro_venta = nro_venta + 1
        if nro_venta < cantidad_ventas:
            input(" Presione ENTER para continuar...")

    return [matriz_ventas, dnis_ventas]

# =====

```

```
# PROGRAMA PRINCIPAL
# =====

# Cantidad de ventas a procesar (cambiando este numero el programa
# sigue funcionando: con 3, con 100, etc.)
CANTIDAD_VENTAS = 3

# 1) Procesamos todas las ventas y obtenemos la matriz 3x4 y la lista de DNIs
resultado_ventas = procesar_todas_las_ventas(CANTIDAD_VENTAS)
matriz_ventas = resultado_ventas[0]
dnis_ventas = resultado_ventas[1]

# 2) Mostramos el cierre (matriz ordenada, estadísticas, buscador y despedida)
mostrar_cierre(matriz_ventas, dnis_ventas)
```